

Interview Questions — Java - Spring WebServices

Total Questions: 25

1. What is an API, and why would a Java application use an AI API such as OpenAI or Google Vertex AI?

Hint: Think about how external services provide functionality to your app.

2. Name two popular AI APIs that can be accessed from Java and mention one use case for each.
-

3. What is the purpose of making a REST call from Java to an AI service like OpenAI's API?

Hint: Consider what happens when your app needs to generate or analyze text using AI.

4. Explain the role of the Java HttpClient when working with REST APIs.
-

5. How would you use HttpClient in Java to make a GET request to an external API? Briefly describe the steps.

Hint: Focus on creating the client, building the request, sending it, and handling the response.

6. What is the Jackson ObjectMapper used for in Java REST API integrations?

Hint: Think about converting between JSON and Java objects.

7. Suppose you receive a JSON response from an API containing a field called 'quote'. Write a simple Java code snippet using Jackson ObjectMapper to extract the value of 'quote' from the JSON string.

Hint: You might use ObjectMapper's readTree method and access the field.

8. How would you handle errors or unexpected responses when making API calls in Java?

Hint: Consider exceptions, status codes, or response validation.

9. Describe how you would fetch an AI-powered 'quote of the day' from a public API and display it in your Java application.
-

10. What are the main steps to integrate the OpenAI API with a Spring Boot application?

Hint: Think about configuration, authentication, and making the API call.

11. Why is it important to keep your OpenAI API key secure when integrating with Spring Boot?

Hint: Consider what could happen if someone else gets access to your key.

12. In a Spring Boot project, how would you create a REST endpoint that accepts a user question and returns a response from the OpenAI API?

Hint: Think about controllers, request mapping, and service classes.

13. Write a Spring Boot controller method that receives a POST request with a question and returns the OpenAI API's answer as the response.

14. How would you test if your 'Ask CampusBot' REST endpoint is correctly calling the OpenAI API and returning the expected output?

15. What is LangChain4J, and how does it help when building AI chatbots in Java?

Hint: Think about how it interacts with language models.

16. Explain what a prompt template is in the context of LangChain4J and why it is useful.

17. Describe the process of creating a simple FAQ chatbot using LangChain4J and OpenAI API in a Java application.

18. If you want your CampusBot to answer college-specific FAQs, how would you structure the prompt templates in LangChain4J?

19. How does LangChain4J simplify chaining multiple prompts or API calls for a chatbot?

Hint: Consider how it manages sequences of operations.

20. Suppose your chatbot is returning irrelevant answers from the OpenAI API. What steps would you take to debug and improve the responses?

Hint: Think about prompt engineering, logging, and API parameters.

21. Compare the process of calling the OpenAI API directly via HttpClient and using LangChain4J as a wrapper. What are the advantages of each approach?

22. How would you use an AI-powered code assistant like GitHub Copilot or ChatGPT to help you write or debug your Java code that integrates with a REST API? What would you do to ensure the code is correct before using it in your project?

Hint: Think about reviewing, testing, and understanding the AI-generated code.

23. Imagine you want to quickly generate prompt templates or sample chatbot responses for your Java project. How could an AI tool help, and what would you check before using its suggestions?

24. Integrate your knowledge: Describe how you would build a Spring Boot endpoint that takes a user question, sends it to OpenAI via LangChain4J, and returns the answer to the user.

25. Suppose you want to log every question and answer exchanged between users and your CampusBot chatbot. Which parts of your Java application would you modify, and how would you implement this logging?

Hint: Think about controller, service, and possible use of a database or log files.